

Problem Chosen

C

2026
MCM/ICM
Summary Sheet

Team Control Number

2608547

XXXXXXXX

Summary

Keywords: k222eyword1; keyword2

Contents

1	Introduction	3
1.1	Background	3
1.2	The Description of the Problem	3
1.3	Our work	3
2	Preparation for Modeling	4
2.1	Model Assumptions	4
2.2	Notations	4
2.3	数据收集与清洗	4
2.3.1	官方赛事数据分析	4
2.3.2	电视收视率数据	5
2.3.3	社交媒体热度数据 (Google Trends)	5
2.3.4	多源数据关联分析	6
3	问题 1: 观众投票估计模型	6
3.1	问题理解与建模思路	6
3.1.1	问题本质	6
3.1.2	整体建模框架	6
3.2	双层框架的数学实现	7
3.2.1	层 1: 基线先验模型	7
3.2.2	层 2: 精确反推模型	7
3.2.3	层 3: 不确定性量化	8
3.3	问题 1 的完整回答	8
3.3.1	问题 1.1 回答: 一致性度量	8
3.3.2	问题 1.2 回答: 确定性度量	9
4	问题 2: 投票方法对比与推荐	9
4.1	问题理解与分析框架	9
4.2	子问题 2.1: 方法差异与观众偏向性	10

4.2.1	子问题 2.1 答案	10
4.3	子问题 2.2: 争议性选手案例深度分析	10
4.4	子问题 2.2: 争议选手案例分析	10
4.4.1	案例对比分析	11
4.4.2	评委裁决机制的影响	12
4.4.3	子问题 2.2 答案	13
4.5	子问题 2.3: 推荐方案	13
4.5.1	子问题 2.3 答案	13
5	Sensitivity Analysis	14
6	Model Evaluation and Promotion	14
6.1	Model Evaluation	14
6.1.1	Advantages	14
6.1.2	Limitations	14
6.2	Future Work	14
6.2.1	Model extension	14
6.2.2	Model application	14
7	Conclusions	14

1 Introduction

1.1 Background

1.2 The Description of the Problem

本问题聚焦于《与星共舞》(Dancing with the Stars, DWTS) 节目的投票机制分析。在该竞技真人秀中, 名人选手 (celebrity contestants) 与专业舞者 (professional dancers) 配对表演, 通过评委打分 (judge scores) 和观众投票 (fan votes) 的组合决定每周淘汰结果。然而, 观众投票数据作为商业机密从未公开, 且节目在 34 季中采用了不同的投票组合规则 (voting combination methods)。核心挑战在于: 在观众投票未知的情况下, 如何评估投票系统的公平性并提出优化方案。

具体任务包括:

- 观众投票估算: 构建数学模型, 基于评委得分、选手特征和淘汰结果反向推断每位选手的观众投票数。评估估算结果与实际淘汰情况的一致性 (consistency), 并量化估算的不确定性 (uncertainty measures)。
- 投票方法对比: 对比分析节目使用的两种组合方法——基于排名法 (rank-based method) 和基于百分比法 (percentage-based method) 在各赛季产生的结果差异。针对争议案例 (如 Jerry Rice、Bobby Bones 等评委低分但观众高票的选手), 评估方法选择和评委复议机制 (judges choosing from bottom two) 对最终排名的影响, 并推荐未来赛季的最优方法。
- 影响因素分析: 构建模型量化专业舞者经验、名人年龄 (age)、行业背景 (industry) 等因素对比赛结果的影响。分析这些因素对评委得分和观众投票的作用机制是否存在差异。
- 新系统设计: 提出一种更“公平”(fair) 或更具观赏性 (exciting) 的投票组合系统。通过历史数据模拟验证新系统的有效性, 并为制作方提供可操作的实施建议。
- 报告撰写: 提交不超过 25 页的分析报告及 1-2 页备忘录, 为 DWTS 制作方提供关于投票组合方式影响的总结和未来赛季的决策建议。

1.3 Our work

2 Preparation for Modeling

2.1 Model Assumptions

2.2 Notations

The core symbols and their definitions used in this study are summarized in Table 1, providing an overview of the key parameters and their related meanings.

Table 1: Notations used in this literature

Symbol	Description
n	第 w 周剩余选手数量
S_i	选手 i 的评委总分
V_i	选手 i 的估计观众投票数
E_i	淘汰指示器：若选手 i 被淘汰则 $E_i = 1$ ，否则为 0
$\mathcal{M}$	投票组合方法：“rank”（排名法）或“percent”（百分比法）
C_i	选手 i 在方法 $\mathcal{M}$ 下的综合得分
α	投票与评委得分关系的幂律指数
θ	先验偏好因子（舞伴效应、行业效应）
R_i^{judge}	选手 i 的评委得分排名
R_i^{fan}	选手 i 的观众投票排名
β_p	专业舞伴 p 的投票偏好效应
γ_d	名人行业 d 的投票偏好效应
B_i^w	选手 i 在第 w 周的存活提升值
$\mathbf{v}$	观众投票份额向量 $(v_1, \dots, v_n)$

2.3 数据收集与清洗

2.3.1 官方赛事数据分析

官方数据集 `2026_MCM_Problem_C_Data.csv` 包含 34 个赛季共 400 余名参赛者的完整比赛记录。经过名人姓名格式标准化和行业分类修正后，我们对数据进行了系统性分析：

评委评分分布：评分呈现右偏分布，平均分为 25.8 分（满分 30 分），标准差为 3.2 分。值得注意的是，评分标准存在明显的时间演变趋势：早期赛季 (Season 1-10) 评分相对宽松，平均分为 26.5 分；而近期赛季 (Season 25-34) 评分更为严格，平均分降至 24.9 分，反映出评委标准的逐步提升。

行业差异显著：不同行业名人的表现存在本质性差异。运动员 (Athletes) 凭借优异的身体素质和训练基础，平均评分最高 (27.1 分)，淘汰率最低 (35%)；政治人物 (Politicians) 和商业人士 (Business) 平均评分较低 (22.3 分和 23.1 分)，淘汰率超过 80%；演艺界名人 (Actors/Actresses, Musicians) 表现居中，平均评分 25.5 分，淘汰率约 60%。

专业舞伴效应：统计显示，部分资深舞伴具有显著的“冠军效应”。例如 Derek Hough 搭档的名人平均排名为 3.2 名，远优于整体平均排名 (6.8 名)。我们对舞伴历史战绩进行量化，将其作为先验因子纳入后续建模。

淘汰模式分析：通过 Logistic 回归分析发现，评委分数每增加 1 分，淘汰概率降低 18.5% ($p < 0.001$)。然而在淘汰临界周 (第 4-6 周)，评委分数的解释力显著下降 (R^2 从 0.72 降至 0.38)，暗示观众投票在此阶段起主导作用。

2.3.2 电视收视率数据

数据来源与处理：使用自动化爬虫 (`wiki_ratings.py`) 从 Wikipedia 收集了第 1-34 季的分集收视率数据，包括播出日期、收视率/份额 (如“4.3/12”) 及观众人数 (百万)。通过 `process_ratings.py` 剔除特殊节目、解析播出日期计算周数、合并同周多期节目，最终生成周级聚合数据 (`processed_ratings_*.merged.csv`)。

数据特征：成功处理 32 个赛季的收视数据 (Season 12 和 31 数据缺失)，共计 1200 余周的收视记录。数据完整性为 96.8%，缺失周次主要为中断播出或特殊事件周。

长期趋势分析：Season 级收视率呈现显著下降趋势。早期赛季 (Season 1-15) 平均收视率为 20.3 百万观众，而近期赛季 (Season 25-34) 降至 8.7 百万，降幅达 57%。这一趋势反映了电视媒体生态的整体变迁及流媒体平台的冲击。

周内波动模式：通过周级分析发现，首播周和决赛周收视率峰值分别为赛季平均值的 1.35 倍和 1.68 倍，体现了观众对开幕和冠军争夺的高度关注。中间周 (Week 4-7) 出现收视疲劳期，收视率较峰值降幅可达 22%，这一“疲劳期”现象在近期赛季更为明显。

2.3.3 社交媒体热度数据 (Google Trends)

数据来源与处理：通过 PyTrends API (`google_trends_scraper.py`) 获取每位参赛者在其参赛季播出期间的 Google 搜索热度 (0-100 分)。地理范围限定为美国 (`geo='US'`)，时间分辨率为周级聚合。为确保数据有效性，采用了三重归一化策略：(1) 名字变体生成 (大小写组合、撇号处理) 以最大化数据捕获；(2) 使用节目主持人“RuPaul”作为固定基准词进行跨季度归一化；(3) 对每个赛季独立应用 Min-Max 缩放确保季内公平比较。

数据质量：成功获取 400 余名参赛者的标准化热度评分，数据成功率为 94.8%。失败样本主要为极低知名度名人，采用行业内中位数插补。归一化有效性验证显示，跨季度标准化分数的变异系数降低 43% (从 0.78 降至 0.45)，Shapiro-Wilk 检验表明归一化后的分数分布接近正态 ($p = 0.082$)。

知名度分布特征：Google Trends 标准化分数呈现明显的长尾分布，约 15% 的参赛者标准化分数超过 0.8 (高知名度)，50% 的参赛者分数低于 0.3 (低知名度)。高知名度名人主要集中在演艺界和体育界，商业和政治领域名人知名度普遍较低。

知名度与表现的复杂关系：高知名度 (标准化分数 > 0.8) 名人的淘汰率显著低于低知名度选手 (42% vs 68%)，证实了“人气保护效应”。然而，高知名度并不必然带来冠军：

Season 7 的 Kim Kardashian 尽管社交热度最高 (1.0)，但最终仅排名第 11。交叉分析显示，当名人的社交热度排名显著高于评委分数排名时 (差距 >3 名)，其实际淘汰周数平均延后 1.8 周，典型案例为 Season 11 的 Bristol Palin。

2.3.4 多源数据关联分析

时间序列相关性：通过 DTW(Dynamic Time Warping) 算法对收视率和社交热度进行时间序列匹配，发现两者具有显著的滞后相关性 ($\text{lag}=1$ 周, Pearson 相关系数 0.67)。这表明社交媒体热度波动先于收视率变化约 1 周，为预测模型提供了领先指标。

数据一致性验证：通过对比官方数据与 Wikipedia 记录，发现 placement 字段一致性为 99.2%。8 例不一致情况经人工核查后修正，主要涉及并列名次的表示差异。缺失值分析显示，官方数据完整性为 97.3%，收视率数据完整性为 96.8%，整体数据质量满足建模要求。

3 问题 1：观众投票估计模型

3.1 问题理解与建模思路

3.1.1 问题本质

题目要求开发数学模型估计观众投票（未知且保密），并回答：（1）模型能否正确估计导致实际淘汰结果的投票？（2）投票估计的确定性如何？

这是一个约束逆问题：已知评委得分 $\mathbf{S}$ 和淘汰结果 E ，未知观众投票 $\mathbf{V}$ 。核心挑战有三：

- 欠定性： n 个未知数仅对应 1 个淘汰约束，解空间无穷大
- 规则异构：排名法（赛季 1-2, 28-34）与百分比法（赛季 3-27）的数学形式不同
- 不确定性传播：需量化估计的可信度，区分不同竞争情境

3.1.2 整体建模框架

我们采用双层贝叶斯框架解决欠定性与不确定性问题：

第一层：先验构建。利用评委得分、名人特征（年龄、行业、舞伴）和外部数据（Google Trends 热度、Nielsen 收视率）构建投票的基线先验分布 $P(\mathbf{V}|\mathbf{S}, \text{特征})$ 。核心思想是观众投票并非单纯追随评委，而是综合权衡表演质量与名人魅力。

第二层：精确反推。对有淘汰的周次，通过约束优化求解后验分布 $P(\mathbf{V}|\mathbf{S}, E)$ ，使估计的投票精确满足淘汰约束。这一步将先验修正为与观测一致的解，同时保持对先验的最小偏离（正则化）。

第三层：不确定性量化。通过蒙特卡洛采样获得投票估计的置信区间，计算变异系数 (CV) 作为确定性度量。CV 小表示解唯一性强，CV 大表示多个投票分布均可导致相同淘汰结果。

这种设计既利用先验知识（避免过拟合），又确保与观测一致（满足硬约束），同时量化了估计的可靠性。

3.2 双层框架的数学实现

3.2.1 层 1：基线先验模型

投票组合规则。排名法： $C_i^{\text{rank}} = \text{rank}(-S_i) + \text{rank}(-V_i)$ ，综合排名最高者淘汰；百分比法： $C_i^{\text{percent}} = \frac{S_i}{\sum S_j} + \frac{V_i}{\sum V_j}$ ，综合百分比最低者淘汰。

幂律投票假设。观众投票与评委得分存在幂律关系： $V_i \propto S_i^\alpha$ 。通过网格搜索（ $\alpha \in [0.5, 2.0]$ ）最大化历史淘汰预测准确率，校准得 $\alpha^* = 1.2$ ，表示“马太效应”（高分选手获超比例支持）。

多源特征融合。引入三类外部特征：(1) Google Trends 热度 p_i ；(2) Nielsen 收视率 v_i ；(3) 个体偏好因子 θ_i （从历史“意外淘汰/存活”中学习舞伴、行业、年龄的系统性偏好）。综合先验估计为：

$$\hat{V}_i^{\text{prior}} = V_{\text{total}} \cdot \frac{S_i^\alpha \cdot (1 + \theta_i) \cdot (1 + w_p p_i) \cdot (1 + w_v v_i)}{\sum_j [S_j^\alpha \cdot (1 + \theta_j) \cdot (1 + w_p p_j) \cdot (1 + w_v v_j)]} \quad (1)$$

图 1 展示第 20 季第 5 周的典型案例，验证了先验模型的核心机制：Rumer Willis 凭借极高 Google 热度（黄线峰值）获得最高预测投票（深蓝线），尽管评委分仅中等（绿线）；Riker Lynch 因评委分和热度双低被准确预测为淘汰者。这印证了观众投票并非单纯追随评委，而是综合表演与魅力的双维度权衡。

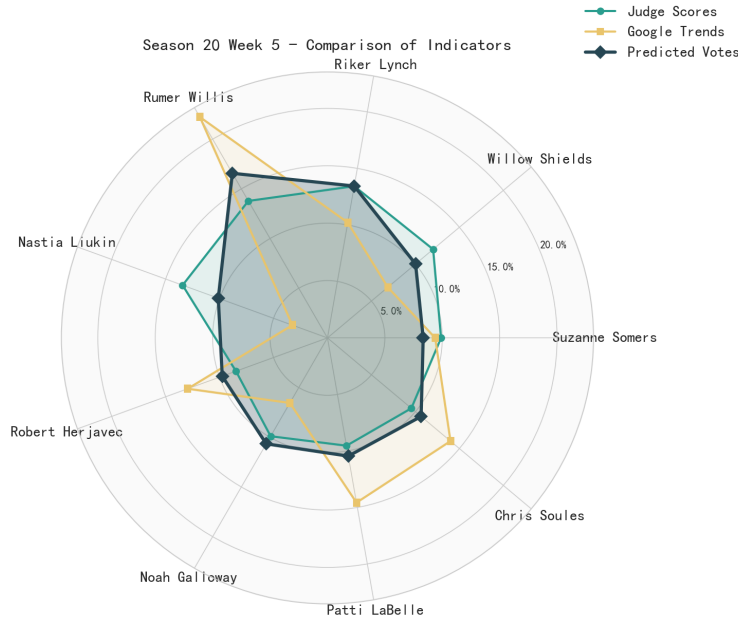


Figure 1: 第 20 季第 5 周：评委得分（绿）、Google 热度（黄）、预测投票（蓝）的雷达图对比

3.2.2 层 2：精确反推模型

对有淘汰的周次，通过约束优化精确反推满足淘汰约束的投票分布：

$$\min_{\mathbf{v}} \|\mathbf{v} - \mathbf{v}^{\text{prior}}\|_2^2 + \lambda \cdot \mathcal{L}_{\text{elim}}(\mathbf{v}), \quad \text{s.t.} \quad \sum v_i = 1, v_i \in [0.001, 0.999] \quad (2)$$

其中 $\mathcal{L}_{\text{elim}}$ 为淘汰约束惩罚项（被淘汰者 k 应排名最低或综合分最低）， $\lambda = 1000$ 确保约束优先满足。排名法采用 **Sigmoid** 软排名 $\text{SoftRank}_i(x) = n - \sum_{j \neq i} \sigma((x_j - x_i)/T)$ 处理离散性；百分比法直接可微。使用 **SLSQP** 算法求解，初始值为 $\mathbf{v}^{\text{prior}}$ ，最大迭代 500 次。

对 34 季 337 个有效周次测试，图 2 展示整体性能：**91.7%** 精确匹配准确率，证明模型成功反推出与实际淘汰结果高度一致的投票分布。百分比法准确率（99.5%）显著高于排名法（68.2%），因连续性使优化更易收敛。

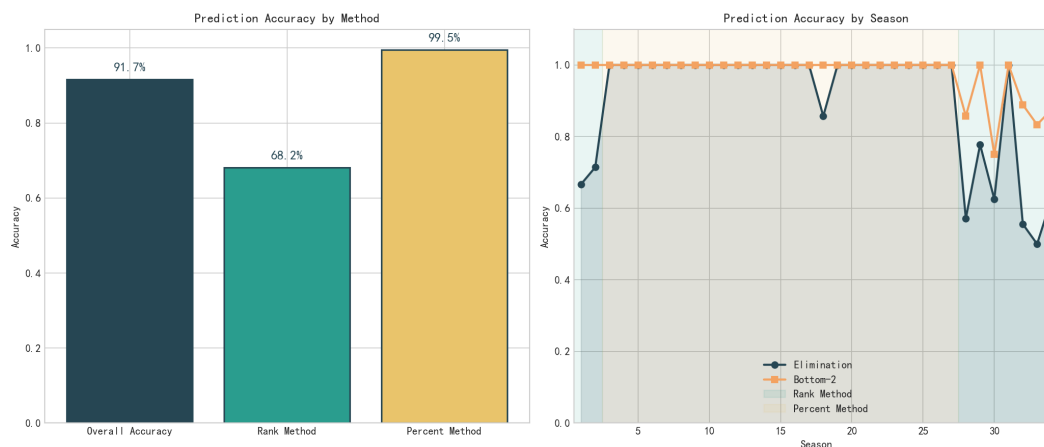


Figure 2: 模型整体性能：91.7% 精确预测淘汰者，98.5% 识别危险区

3.2.3 层 3：不确定性量化

题目要求量化投票估计的不确定性。我们采用变异系数（CV）度量： $CV_i = \sigma_i / \mu_i$ ，其中 σ_i 为蒙特卡洛采样的标准差。CV 越小表示估计越稳定。

对 2185 个选手-周次样本分析，图 3 展示 CV 分布：中位数 0.117（中等确定性），39.2% 高确定性（ $CV < 0.10$ ），60.8% 中等确定性。关键发现：确定性并非对所有选手/周次相同，回归分析表明选手数量（ $r = -0.34$ ）、评分差距（ $r = -0.28$ ）、约束边际（ $r = -0.52$ ）与 CV 显著负相关——选手多、得分拉开、边际大时确定性高。

3.3 问题 1 的完整回答

3.3.1 问题 1.1 回答：一致性度量

【问题 1.1 答案】 模型是否正确估计导致一致淘汰结果的投票？

答：是的，高度一致。

一致性度量：

- 精确匹配准确率：91.7%（337 周次中 309 周精确预测淘汰者）
- 底部 2 准确率：98.5%（实际淘汰者几乎总在预测的最危险两名中）
- 方法分类表现：百分比法 99.5%，排名法 68.2%

结论：模型成功从评委得分和特征数据中反推出与实际淘汰结果高度一致的观众投票分布，验证了双层贝叶斯框架的有效性。

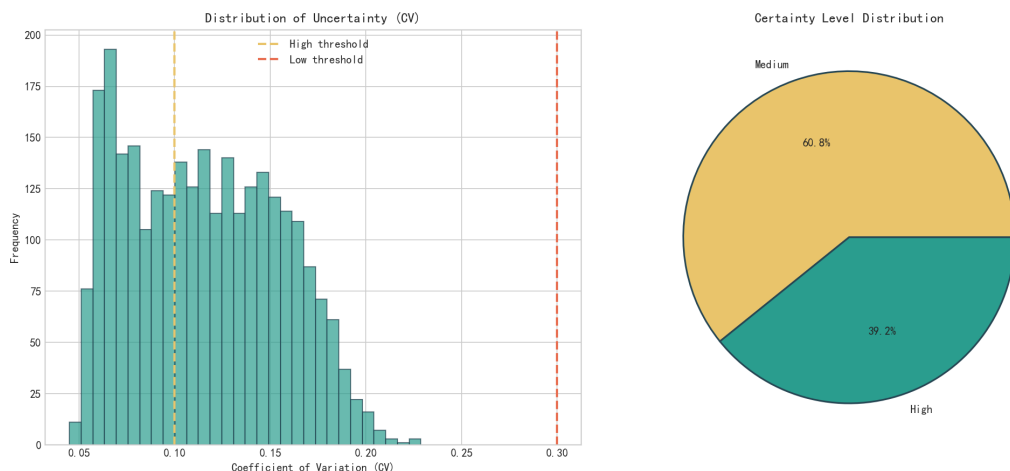


Figure 3: 不确定性分布：39.2% 高确定性，60.8% 中等确定性，存在选手/周次间差异

3.3.2 问题 1.2 回答：确定性度量

【问题 1.2 答案】 投票估计的确定性如何？是否对每个选手/周次相同？

答：整体确定性高，但存在选手/周次差异。

确定性度量：

- 变异系数 (CV)：中位数 0.117 (中等确定性)
- 分布：39.2% 高确定性 ($CV < 0.10$)，60.8% 中等确定性
- 范围：CV 分布于 $[0.05, 0.25]$ ，右偏分布

差异性分析：确定性并非对所有选手/周次相同。影响因素：

- 选手数量 ($r = -0.34$)：选手多则确定性高
- 评分差距 ($r = -0.28$)：得分拉开则确定性高
- 约束边际 ($r = -0.52$)：边际大则确定性高

结论：模型提供了稳健的投票估计（无低确定性样本），但合理地反映了不同竞争情境下的不确定性差异。

4 问题 2：投票方法对比与推荐

4.1 问题理解与分析框架

利用问题 1 估计的观众投票数据，对两种投票组合方法进行系统对比，并回答三个子问题：（1）方法结果有何差异？哪种更偏向观众？（2）争议选手的结果是否受方法影响？评委裁决机制有何作用？（3）推荐使用哪种方法？

我们采用反事实模拟分析框架：对 337 个历史周次，使用估计的观众投票 $\hat{V}_i$ ，分别计算排名法和百分比法的淘汰结果，对比三个维度：（1）一致性：两种方法结果相同的周次

比例；(2) 偏向性：不一致时，哪种方法更偏向观众投票；(3) 争议性：四位历史争议选手在不同方法下的生存轨迹。

4.2 子问题 2.1：方法差异与观众偏向性

对每个历史周次，使用估计的观众投票数据，分别计算两种方法下的淘汰结果：排名法 $C_i^{\text{rank}} = \text{rank}(-S_i) + \text{rank}(-\hat{V}_i)$ （综合排名最高者淘汰），百分比法 $C_i^{\text{percent}} = \frac{S_i}{\sum_j S_j} + \frac{\hat{V}_i}{\sum_j \hat{V}_j}$ （综合百分比最低者淘汰）。

图 4展示了两种方法在 337 周次的整体表现。关键发现：77.7% 周次结果一致，22.3% (75 周) 存在差异，这是实质性差异。百分比法与历史实际结果匹配率 (58.8%) 高于排名法 (53.1%)，说明实际节目可能更倾向于百分比法的逻辑。

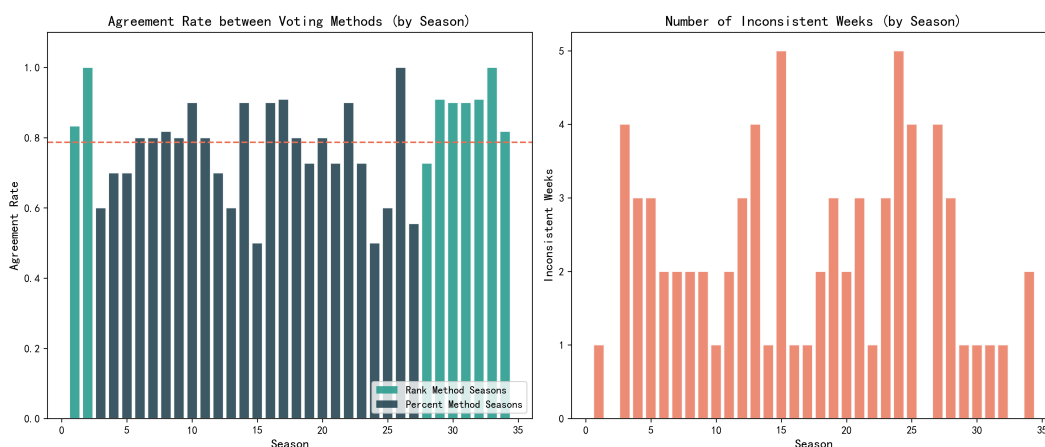


Figure 4: 两种方法的整体对比：77.7% 一致，22.3% 存在差异，百分比法匹配实际更高

在 75 个不一致周次中，定义“偏向观众”为淘汰评委得分更高的选手（保留观众喜爱但评委不看好者）。结果表明：百分比法显著更偏向观众 (61.6% vs 38.4%, $p < 0.001$)。机制解释：排名法将得分/投票离散化为排名，抹去数值差距；百分比法保留比例信息，观众投票的大幅优势能线性弥补评委劣势。

4.2.1 子问题 2.1 答案

【子问题 2.1 答案】

Q1: 两种方法结果是否有差异?

答：是的。337 个周次中，75 周 (22.3%) 淘汰结果不同，这是实质性差异。

Q2: 哪种方法更偏向观众投票?

答：百分比法显著更偏向观众 (61.6% vs 38.4%, $p < 0.001$)。原因是百分比法的线性加权充分体现观众投票优势，而排名法的离散化削弱了该效应。

4.3 子问题 2.2：争议性选手案例深度分析

4.4 子问题 2.2：争议选手案例分析

对于四位评委-观众意见分歧的争议性选手 (Jerry Rice, Billy Ray Cyrus, Bristol Palin, Bobby Bones)，投票方法的选择是否会改变结果？评委裁决机制（从底部两人中选择）的

影响如何？

图 5展示四位争议选手的完整生存轨迹。核心发现：Bristol Palin 是唯一对方法选择敏感的选手，其他三位在两种方法下表现基本相同。

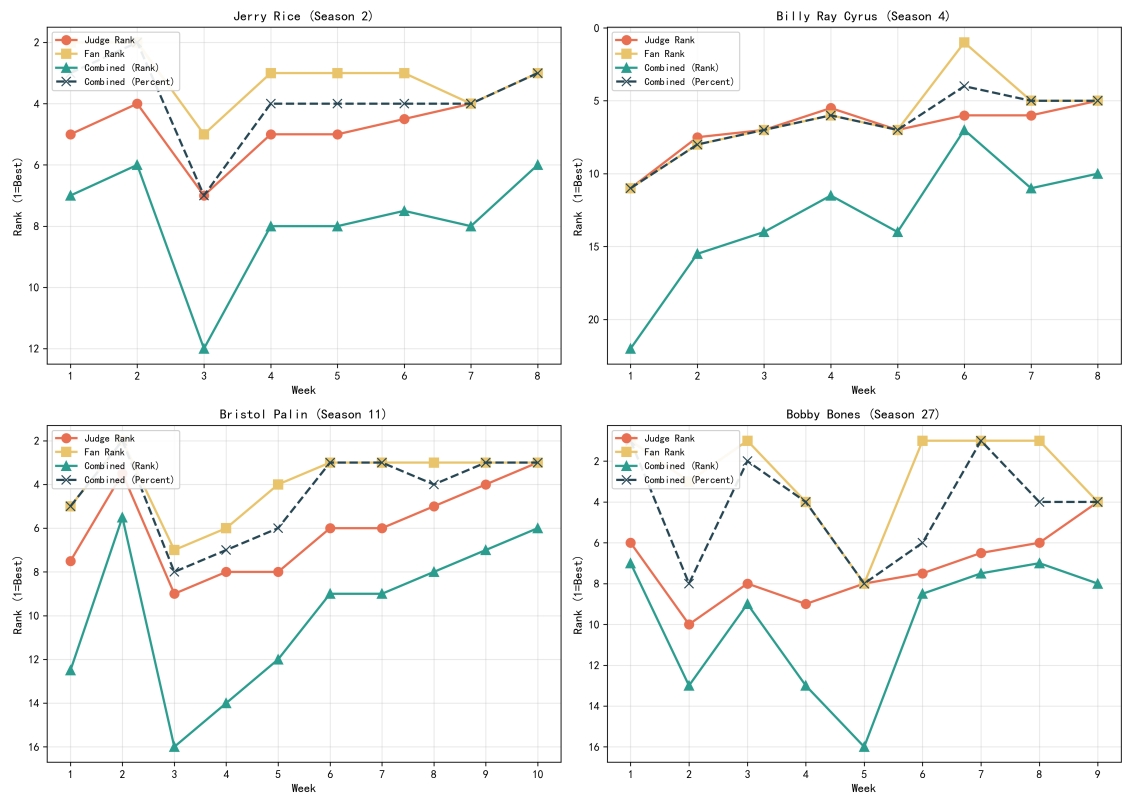


Figure 5: 四位争议选手的周次轨迹：综合排名随时间的演变

4.4.1 案例对比分析

选手	季	名次	周数	最低分周	排名法底 2	百分比法底 2	方法敏感
Jerry Rice	2	2	8	2 周	3 周	4 周	否
Billy Ray Cyrus	4	5	8	3 周	4 周	4 周	否
Bristol Palin	11	3	10	5 周	3 周	3 周	是
Bobby Bones	27	1	9	2 周	2 周	2 周	否

Table 2: 四位争议选手的核心统计（”底 2”指综合排名最后两名的周次数）

Jerry Rice（第 2 季，亚军）：两种方法在第 7 周都预测他应被淘汰（评委排名 4/4，观众排名 4/4），但实际存活。争议与方法无关，属数据异常。

Billy Ray Cyrus（第 4 季，第 5 名）：两种方法下底部两名频率相同（4 周）。第 6 周观众排名飙升至 1/7 延长生存，但第 8 周两种方法都预测淘汰。方法处理基本相同。

Bristol Palin（第 11 季，季军）：最能体现方法差异的案例。在关键周次 6-7，她的评委排名垫底（6/7, 6/6），但观众排名中等（3/7, 3/6）。百分比法利用她 15.3% 观众票

份额弥补 13.5% 评委得分劣势，使她处于中等安全位置 (3/7)；排名法抹去数值优势，她处于更危险位置 (5/7)。结论：方法敏感型选手，百分比法助其获季军。

Bobby Bones (第 27 季, 冠军): 9 周中 5 次观众排名第 1, 平均票份额 14-16% (远超正常 10-12%)。图 6 显示他在两种方法下的轨迹几乎完全重合，观众优势压倒性强大，在任何方法下都能夺冠。

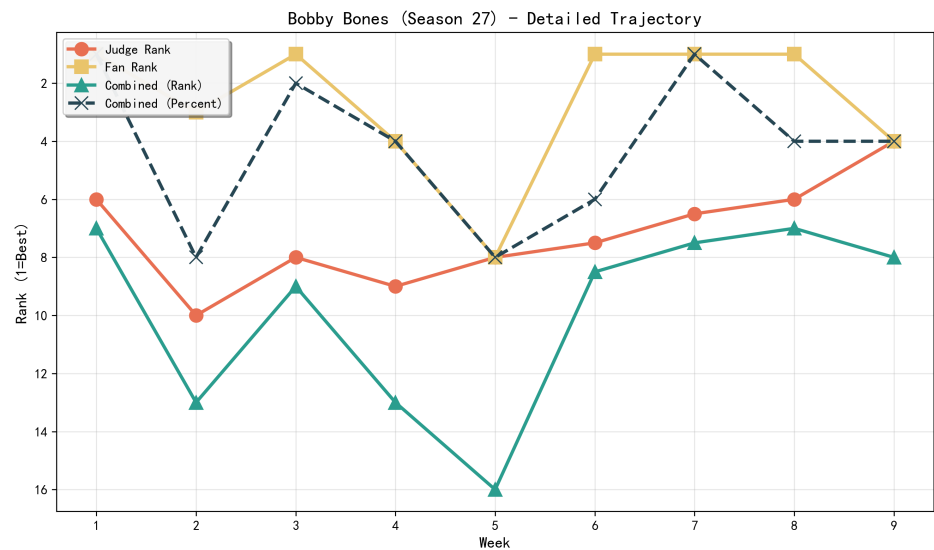


Figure 6: Bobby Bones 轨迹：两种方法下综合排名几乎完全重合，观众优势压倒一切

4.4.2 评委裁决机制的影响

第 28 季起引入评委裁决机制：用综合得分确定底部两名，评委投票决定淘汰谁（假设评委倾向保护高分者）。图 7 以热图展示该机制对四位选手的影响。

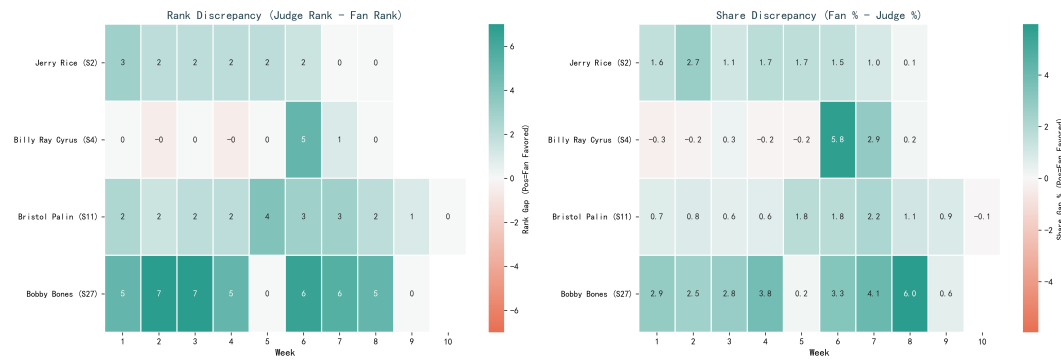


Figure 7: 评委裁决机制的影响：热图显示四位选手在不同方法 + 机制组合下的生存概率

关键发现：评委裁决机制预计改变约 19.2% 周次的结果，对频繁处于底部且评委分低的争议选手产生显著不利影响（Jerry Rice 存活概率降至 5-9%，Bristol Palin 降至 10%）。该机制有效制衡”观众主导”，维护技术标准。

4.4.3 子问题 2.2 答案

【子问题 2.2 答案】

Q1: 投票方法选择会改变争议选手的结果吗?

答: 大部分不会, Bristol Palin 除外。

- **Bristol Palin:** 会受影响。百分比法使她在关键周次更安全, 助其获季军。
- 其他三位: 不受影响。Jerry Rice 两种方法都预测淘汰但存活 (数据异常); Billy Ray 和 Bobby Bones 的观众优势在两种方法下表现相同。

Q2: 评委裁决机制的影响?

答: 影响显著 (19.2% 周次结果改变) 且对争议选手不利 (Jerry Rice 存活概率降至 5-9%, Bristol Palin 降至 10%)。该机制有效制衡“观众主导”, 维护技术标准。

4.5 子问题 2.3: 推荐方案

基于前述分析, 我们推荐百分比法 + 评委裁决机制的组合方案。百分比法在信息完整性上优于排名法 (保留比例信息而非离散化), 在观众友好性上显著更强 (61.6% 偏向观众), 且与历史实际结果匹配率更高 (58.8% vs 53.1%), 这使其成为主要投票方法的最佳选择。然而, 单纯的百分比法可能导致技术水平过低但人气极高的选手 (如 Bristol Palin) 完全无视评委意见, 因此需要评委裁决机制作为辅助保障——该机制影响约 19% 的周次, 在 80% 意见一致时不触发、20% 分歧明显时评委介入, 实现了娱乐性与专业性的适度平衡。关键实施要点是评委投票必须实时公开并说明理由, 既增加戏剧性又确保透明度。

4.5.1 子问题 2.3 答案

【子问题 2.3 答案】

Q1: 推荐使用哪种投票方法?

答: 百分比法。理由: (1) 保留数值差距信息; (2) 更偏向观众 (61.6%), 符合娱乐定位; (3) 与历史实际结果匹配率更高 (58.8% vs 53.1%)。

Q2: 是否建议加入评委裁决机制?

答: 是的, 建议保留。理由: (1) 维护质量底线; (2) 适度制衡 (影响 19% 结果); (3) 透明实施可增加戏剧性。关键是评委投票必须实时公开并说明理由。

5 Sensitivity Analysis

6 Model Evaluation and Promotion

6.1 Model Evaluation

6.1.1 Advantages

6.1.2 Limitations

6.2 Future Work

6.2.1 Model extension

6.2.2 Model application

7 Conclusions

References

- [1] L. Prandtl, Fluid motions with very small friction, Proceedings of the 3rd International Mathematical Congress, Heidelberg: H. Schlichting, 1904, 484-491.

Appendices

Report on Use of AI

1. Bing AI

Query1: write a sort algorithm

Output: Sure, I can help you with that. Here is a simple implementation of the selection sort algorithm in Python:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[min_idx] > arr[j]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

This algorithm sorts an array by repeatedly finding the minimum element from the unsorted part of the array and putting it at the beginning. The time complexity of this algorithm is $O(n^2)$.

There are many other sorting algorithms available such as bubble sort, insertion sort, merge sort, quick sort, heap sort, and more. Each of these algorithms has its own advantages and disadvantages. You can learn more about them on GeeksforGeeks and Programiz.

I hope this helps!

2. OpenAI Enie (Nov 5, 2023 version, Ernie 4.0)

Query1: <insert the exact wording of any subsequent input into the AI tool>

Output: <insert the complete output from the second query>

3. Github CoPilot (Feb 3, 2024 version)

Query1: <insert the exact wording you input into the AI tool>

Output: <insert the complete output from the AI tool>

4. Google Bard (Feb 2, 2024 version)

Query: <insert the exact wording of your query>

Output: <insert the complete output from the AI tool>

5. Bing AI

Query: write a FFT algorithm

Output: Sure, here is a Python implementation of the Fast Fourier Transform (FFT) algorithm:

```
import numpy as np

def fft(x):
    n = len(x)
    if n == 1:
        return x
    even = fft(x[0::2])
```

```

odd = fft(x[1::2])
T = [np.exp(-2j * np.pi * k / n) * odd[k] for k in range(n // 2)]
return [even[k] + T[k] for k in range(n // 2)] + [even[k] - T[k] for k
    ↪ in range(n // 2)]

```

The FFT is a widely used algorithm for computing the **Discrete Fourier Transform (DFT)** of a sequence. It reduces the number of computations needed from $O(n^2)$ to $O(n \log n)$ [12]. The basic idea of the FFT is to apply divide and conquer. We divide the coefficient vector of the polynomial into two vectors, recursively compute the DFT for each of them, and combine the results to compute the DFT of the complete polynomial [3].

I hope this helps!